Quantized reward-model verifiers in Best-of-N mathematical answer selection: accuracy, memory and value retention

Hector Borobia[a,*], Elies Seguí-Mas[b], Guillermina Tormo-Carbó[c]

[a] *VRAIN – Valencian Research Institute for Artificial Intelligence, Universitat Politècnica de València, Valencia, Spain*
[b] *Department of Economics and Social Sciences, Universitat Politècnica de València, Valencia, Spain*
[c] *Department of Business Organisation, Universitat Politècnica de València, Valencia, Spain*

Abstract

Reward-model verifiers improve Best-of-N answer selection but compete with the generator for accelerator memory. We evaluate post-training weight quantization of the verifier on frozen pools of 32 solutions per problem from GSM8K and MATH, using three outcome reward models (1.7B–8B) from one family and a 7B process reward model (PRM), with GPTQ, AWQ, NF4 and round-to-nearest. Analysis plans were fixed in version control before verifier scoring, and those of two follow-up studies before candidate generation. On its prespecified 1,200-problem population, the 8B GPTQ-W4 contrast meets a ±3-point equivalence bound for Best-of-32 accuracy (difference +0.33; 90% interval −0.58 to +1.25). Two follow-up studies on unused MATH problems support GPTQ-W4 equivalence for an 8B outcome verifier and the PRM.

*Corresponding author.
Email addresses: hecboar@doctor.upv.es (Hector Borobia), eseguim@upvnet.upv.es (Elies Seguí-Mas), gtormo@omp.upv.es (Guillermina Tormo-Carbó)

Where both verifiers beat majority vote, the PRM retains an estimated 93% of its 8.2-point advantage and passes the prespecified half-retention test; with a stronger generator, no clear verifier advantage was established. Two Llama-based reward models also stay within the bound (losses 0.6–1.5 points). Under a verifier-only 6 GB budget, a packed 4-bit 4B outcome verifier outperforms the tested 1.7B full-precision verifier by 2.25 points under strict grading and 1.40 points after a human audit, with unchanged throughput. Four-bit quantization often changes the selected candidate but rarely its answer; 3-bit GPTQ degrades selection with generic calibration, much less with in-domain calibration. A strict answer grader can turn reward-model format preferences into spurious quantization gains, so verifier compression should be evaluated with format-robust grading and against majority vote.
Keywords: post-training quantization, reward models, Best-of-N, test-time compute, mathematical reasoning, preregistration

---

1. Introduction

Sampling several candidate solutions and letting a learned verifier pick one (Best-of-N) is one of the simplest ways to turn extra inference compute into accuracy on mathematical reasoning [1, 2, 3]. The verifier is usually a reward model of a few billion parameters: an outcome reward model (ORM) that scores a complete response, or a process reward model (PRM) that scores each reasoning step. In deployment it competes with the generator for accelerator memory. A natural way to make room is post-training quantization (PTQ) of the verifier's weights to 4 bits or fewer [4, 5, 6].

Whether this preserves selection quality is not obvious. PTQ is usually

evaluated by perplexity or by the accuracy of the quantized model on its own task, whereas a verifier only needs to *rank* candidates so that a correct one ends up on top. Small score perturbations are harmless if they do not change which answer wins, but Best-of-N is sensitive to errors at the top of the ranking [7]. One might therefore expect quantization damage to grow with the budget N and with problem difficulty, and to matter most for small verifiers. Conversely, if quantized verifiers select as well as full-precision ones, a memory budget may be better spent on a larger quantized verifier.

We study these questions with frozen candidate pools and prospectively specified analysis plans. A pool of 32 candidates per problem is generated once, so every verifier and precision ranks exactly the same responses and differences in accuracy can only come from the verifier. We evaluate three ORM sizes and one PRM in BF16, NF4, GPTQ (4 and 3 bits), AWQ and round-to-nearest on 2,000 GSM8K and MATH problems, measure verifier memory with packed 4-bit kernels, and add two follow-up studies, with a stronger generator and with new hard problems, whose plans were fixed before their candidates were generated.

Our contributions are:

1. **Accuracy equivalence at 4 bits.** GPTQ-W4 meets a registered ± 3 pp equivalence bound for Best-of-32 accuracy of an 8B ORM (90% interval $[-0.58, +1.25]$), and the result holds in two follow-up studies for both an ORM and a PRM. Three-bit GPTQ with generic calibration degrades selection, severely for the smallest verifier.
2. **Retained verifier value.** In a follow-up study on 1,060 new hard problems, where the verifiers beat majority vote, 4-bit quantization

keeps more than half of the advantage (prespecified test): the PRM retains an estimated 93% of an 8.2 pp advantage, the ORM 91% of a 3.0 pp advantage (narrowly significant). With a strong generator, no clear verifier advantage over majority vote was established.

3. **A tested memory comparison.** Under a 6 GB verifier-only budget and a fixed workload, a 4B ORM with packed 4-bit weights outperforms the best tested full-precision verifier that fits by 2.25 pp under the registered grader and 1.40 pp after a blinded human audit of unparseable winners.
4. **Candidate-level instability without accuracy loss.** Quantization changes the selected candidate on a quarter to a half of problems at 4 bits, but the selected answer far less often; harm and benefit stay small and largely offset, consistent with answer redundancy in the pools.
5. **Evaluation lessons.** A strict answer grader can turn reward-model format preferences into spurious quantization gains. We show how a format/content decomposition, a lenient grader, a blinded human audit and revised labels resolve this, and we report two registered hypotheses (damage concentrated on MATH, growing with N) that the evaluation data did not support.

2. Related work

Verifiers and test-time compute. Training a verifier to rank sampled solutions was introduced for grade-school mathematics by Cobbe et al. [1]. Process supervision, which scores every reasoning step, outperforms outcome supervision on MATH [2], and automatically labelled PRMs [8, 9, 10] and generative

verifiers [11] have followed. How to spend inference compute between sampling and verification has become a research question in itself [3, 12, 13]. These works study what verifiers can achieve at full precision; the verifier's own cost enters only as a fixed term.

Reward models against self-consistency. Majority vote over sampled answers (self-consistency) needs no verifier [14], and its coverage and accuracy keep growing with the number of samples [15, 16]. Whether a reward model beats it depends on the generator, the reward model and the aggregation rule [17, 18]; calibrated combinations of answer frequency and PRM scores can outperform both [19]. In a study of memory-optimal test-time strategies for reasoning models, Kim et al. [20] include a Best-of-N experiment with an external 7B PRM counted at full precision (13.28 GB; Section 4.1 of the ICLR 2026 version) and find it less memory-efficient than majority voting. Their main analysis quantizes the generator and excludes verifiers by design. Our study addresses the complementary question: how much of a verifier's cost can be removed by quantizing the verifier itself, and whether its selections survive.

Cheaper verification. Other routes reduce the cost of Best-of-N verification without compressing the reward model: rejecting low-scoring generations early [21], learning lightweight rewards from the generator's hidden states [22], or scoring from the generator's key–value cache instead of re-encoding the text [23]. These change what is computed; weight quantization keeps the verifier and its inputs unchanged and only lowers the precision of its weights, so it can be combined with them.

Reward-model reliability. Reward-model quality is usually measured on preference and step-error benchmarks [24, 25, 26], and Best-of-N is known to over-optimise imperfect reward models [7]. Our question is different: not whether a reward model is accurate, but whether a compressed copy of it selects as well as the original.

Post-training quantization of language models. Post-training quantization reduces memory with little loss of perplexity or task accuracy at 8 bits [27, 28] and, for weights only, at 4 bits [4, 5, 6]; 4-bit precision is close to optimal for a fixed bit budget [29], and large evaluations find that well-calibrated 4- and 8-bit models track their full-precision versions [30]. The topic is active in this journal as well [31, 32]. For reasoning, quantizing the *generator* hurts more at 3 bits and on harder problems [33, 34]. Aggregate accuracy can also hide many changed outputs: compressed models keep their accuracy while flipping a sizeable share of individual answers [35]. We measure the analogue for selection: how often the selected candidate, and how often its answer, changes.

Quantized evaluators. The closest work quantizes LLM-as-judge models: a concurrent workshop submission [36] reports that INT8 judges keep their agreement with human labels within a preregistered equivalence region on pairwise chat comparisons, while NF4 flips a quarter to a third of verdicts and lowers one judge's agreement. We study scalar reward models used for Best-of-N selection on problems with verifiable answers, several quantization methods, a memory comparison with packed kernels, and the verifier's advantage over majority vote. We did not find prior work that quantizes the verifier in Best-of-N and measures the effect on selection.

Evaluation practice. Math benchmark results are sensitive to answer extraction, contamination and label errors [37, 38, 39, 40], and confidence intervals are still uncommon in model evaluations [41]. Preregistration has been proposed for predictive modelling [42] but remains rare in machine learning. We specify the analysis plans prospectively in version control, report uncertainty for every contrast, and check the outcome measure with a lenient grader, a human audit and revised labels.

3. Experimental setup

3.1. Analysis plans

The design, estimands and tests were specified prospectively in a version-controlled repository, with a stated chronology (Table C.5 in Appendix C). For the main study the plan and its amendments were committed after the candidate pools had been generated but before any verifier score on the evaluation problems existed; only generator-side statistics (accuracy, share of extractable answers, truncation) had been inspected at that point. The two follow-up studies were fixed, with their problem manifests and analysis code, before their candidates were generated. We did not use an external registry, so the commit history is the record of what was fixed when. Hypotheses are labelled c1–c5 (main study), H6a/b and H7a/b (follow-up studies); results not covered by these tests are exploratory and labelled as such.

3.2. Problems and candidate pools

Table 1 lists every population. The main evaluation set has 2,000 problems: 800 from the GSM8K test split [1], 400 sampled from MATH500 [2], and 800

further MATH test problems [43] outside MATH500, sampled with a fixed seed. The first 1,200 (GSM8K and MATH500) form the original manifest, frozen before the MATH extension was added, and are the registered population of c1. A development set of 400 problems (200 GSM8K train, 200 MATH train) was used only for the go/no-go gate and for choices assigned to development data.

Table 1: Problem populations. "New" problems appear nowhere else in this work (text-level exclusion against all other items and against all of MATH500).

population	n	composition	used for
Main, full	2,000	800 GSM8K + 400 MATH500 + 800 MATH test	c2, Table 3
Main, original manifest	1,200	800 GSM8K + 400 MATH500	c1
Main, MATH	1,200	400 MATH500 + 800 MATH test	c4, c5
H6	800	MATH test Level 5 (new)	H6a/b
H7	1,060	MATH test Level 4 (905) + Level 5 (155), new	H7a/b

The main generator is Qwen2.5-1.5B-Instruct [44]. For every problem it produces a pool of 32 candidates (4 seeds $\times$ 8 samples, temperature 0.8, top-p 1.0, at most 1,024 new tokens) with the system prompt "Please reason step by step, and put your final answer within `\boxed{}`." Pools are generated once in BF16 with vLLM 0.30 [45], frozen and hashed; every verifier and precision scores exactly the same candidates. Candidate accuracy is 52.1%, 88.7% of candidates have an extractable final answer, and 6.5% reach the length limit. A secondary generator, Llama-3.2-3B-Instruct, produces pools for a 300-problem subset (exploratory; Section 4.7).

3.3. Verifiers

The main study uses three outcome reward models (ORMs) of one family, Skywork-Reward-V2-Qwen3 at 1.7B, 4B and 8B parameters [46], and one process reward model (PRM), Qwen2.5-Math-PRM-7B [10]. The Skywork models are general preference reward models, trained on broad human-preference data rather than on mathematical correctness; we use them as outcome verifiers by scoring the full (problem, response) pair with their scalar head. The PRM is trained for mathematical step correctness; its step rewards are aggregated by their minimum (the last-step reward is reported as a sensitivity analysis). To test generality beyond this family, two further ORMs on a different backbone are evaluated in exploratory experiments: Skywork-Reward-V2-Llama-3.1-8B (same training-data family, Llama 3.1 backbone) and Llama-3.1-Tulu-3-8B-RM [47] (different laboratory and data). All models are run with their documented input formats (Appendix A).

3.4. Precisions and deployments

Quantization is weight-only: the weights of all linear layers inside the transformer blocks are quantized, while embeddings, norms and the reward head stay in BF16, and activations and computation stay in BF16. "BF16" denotes the uncompressed model. The conditions are:

- **NF4**: 4-bit NormalFloat [6], both simulated and deployed with `bitsandbytes` (bit-identical scores);
- **GPTQ-W4** and **GPTQ-W3**: GPTQ [4] with group size 128, calibrated on 128 WikiText-2 sequences of 512 tokens (generic calibration);

- **AWQ-W4** [5] (same calibration) and round-to-nearest **RTN-W4**/**RTN-W8** (exploratory dose-response).

Accuracy experiments use simulated ("fake") quantization: weights are quantized and dequantized to BF16, so only the rounding changes. For the memory study, GPTQ-W4 models are exported and run with packed 4-bit kernels (`gptqmodel`, Marlin). Every quantized model is audited: all target layers must change and no other parameter may change.

Verifier memory is the peak allocated GPU memory, in GiB (2^{30} bytes), while scoring a fixed workload of 64 sequences of 900 tokens on an RTX 4090 with no generator resident.

3.5. Selection rules and baselines

Best-of-N selects the highest-scoring candidate among N; exact score ties are broken by a fixed random priority (seed 11) shared by all conditions. For $N < 32$, accuracy is averaged over 200 random subsets per problem, drawn identically for every condition (seed 7); $N = 32$ is the full pool. Two verifier-free baselines are reported. Majority vote is a plurality vote over canonical answer clusters (numeric normalisation, then symbolic equivalence with `math_verify`): candidates without an extractable answer do not vote, the most frequent cluster wins, ties go to the tied cluster that appears first in candidate order, a pool without any extractable answer counts as incorrect, and the vote is correct if its cluster contains a correct candidate. It does not depend on the verifier. The oracle (pass@32) is correct if any candidate is.

3.6. Grading

The registered grader extracts the last `\boxed{}` answer (else a GSM8K-style "####" line or an explicit "the answer is" phrase), normalises numbers and compares symbolically for MATH. Because a strict grader can confound verifier preferences with answer format, we report three checks: a lenient grader (the last number or expression in the final lines when no answer is extractable; sensitivity only), a blinded human audit (Appendix B), and a re-grading of GSM8K with the revised GSM8K-Platinum labels [38]. The endpoint throughout is final-answer correctness.

3.7. Metrics and controls

For verifier v and condition c, A_N is Best-of-N accuracy and $\Delta_N = A_N^c - A_N^{\mathrm{BF16}}$, computed per problem and averaged. At $N = 32$ we also report answer transitions relative to BF16: the share of problems whose selected candidate changes, whose selected canonical answer changes, whose correct BF16 selection becomes wrong (harm) and the reverse (benefit), so that Δ_{32} equals benefit minus harm. Two controls characterise numerical sensitivity: *bf16-null* rescores in BF16 with a different batch layout, and *bf16-repeat* rescores with the same layout. The latter is bit-identical; the former changes the selected candidate on 7–11% of main-study problems with $|\Delta_{32}| \leq 0.3\,\mathrm{pp}$.

3.8. Statistics

All inference uses a paired percentile bootstrap over problems with 10,000 resamples (seed 7). Problems are resampled with replacement from the full population, and every quantity computed on a problem (both conditions, all

verifiers entering a contrast, the majority-vote outcome) is resampled jointly, so within-problem correlations are kept.

- *Differences* are reported with 95% percentile intervals and two-sided centred-bootstrap p-values.

- *Equivalence* at the registered ± 3 pp margin is tested by two one-sided tests (TOST): $p_{\mathrm{TOST}} = \max\{P^*(\bar{d} \leq -0.03),\, P^*(\bar{d} \geq 0.03)\}$, which inverts the percentile interval ($p_{\mathrm{TOST}} \leq \alpha$ if and only if the $(1 - 2\alpha)$ interval lies inside the margin). In a simulation at the margin ($n = 800$, skewed three-valued per-problem differences, 2,000 replicates) the rejection rate was 0.056, 0.024 and 0.006 at nominal 0.05, 0.025 and 0.0125 (Monte Carlo standard error about 0.005).

- *Value retention* (H6b, H7b) is one-sided: $p = P^*(\bar{R} \leq 0)$ (Section 4.5).

- *Resolution*: when no resample exceeds the boundary we report $p \leq 10^{-4}$ (and $\leq m \cdot 10^{-4}$ after Holm adjustment of the m-th smallest p-value), never $p = 0$.

- *Multiplicity*: Holm correction within each prespecified family (c1–c5; H6a/b; H7a/b); a test of the family that was not run keeps $p = 1$. The registered decision rule for c1 is that its 90% interval lies inside ± 3 pp.

The registered contrasts are: c1, Δ_{32} of the 8B ORM at GPTQ-W4 on the original 1,200-problem manifest (equivalence); c2, the development-selected deployment versus the best BF16 deployment under the 6 GB verifier memory ceiling; c3, a calibration method that was not run; c4, Δ_{32} on the 1,200 MATH problems (4B and 8B ORMs, GPTQ-W4, averaged within problem);

c5, the change of that loss from $N = 4$ to $N = 32$. Table 2 summarises every confirmatory test.

4. Results

Table 2 summarises every confirmatory test; the subsections below give the details and the exploratory analyses around them.

Table 2: All confirmatory tests. Accuracies are Best-of-32 (%) under the registered grader; MV is the verifier-independent majority vote. Estimates in percentage points with 95% intervals (c1: 90%, its registered decision rule). p: raw bootstrap p-value (TOST for equivalence tests, one-sided for H6b/H7b); p_{H}: Holm within the family. $p \leq 10^{-4}$: no resample beyond the boundary.

test	population	contrast	BF16	Q	MV	estimate [CI]	p	p_{H}	result
c1	main, 1,200	8B ORM, W4 vs BF16, ±3	77.5	77.8	78.3	+0.33 [−0.58, +1.25]	$\leq 10^{-4}$	–	equivalent
c2	main, 2,000	4B W4 packed vs 1.7B BF16	71.8	74.1	72.1	+2.25 [+1.05, +3.45]	$\leq 10^{-4}$	≤0.0005	supported
c3	–	calibration method (not run)	–	–	–	–	1	1	not tested
c4	main MATH, 1,200	4B+8B ORM, W4 vs BF16	67.1	66.5	64.8	−0.63 [−1.38, +0.08]	0.099	0.40	not supported
c5	main MATH, 1,200	loss growth N=4→32	–	–	–	−0.51 [−1.20, +0.16]	0.140	0.42	not supported
H6a	H6, 800	8B ORM, W4, ±3	72.8	72.8	76.5	0.00 [−1.13, +1.13]	$\leq 10^{-4}$	≤0.0004	equivalent
H6a	H6, 800	PRM, W4, ±3	76.6	77.1	76.5	+0.50 [−0.50, +1.50]	$\leq 10^{-4}$	≤0.0003	equivalent
H6b	H6, 800	8B ORM, $R > 0$				−1.88 [−3.19, −0.63]	0.998	0.998	not supported
H6b	H6, 800	PRM, $R > 0$				+0.56 [−0.63, +1.69]	0.182	0.365	not supported
H7a	H7, 1,060	8B ORM, W4, ±3	59.0	58.7	55.9	−0.28 [−1.42, +0.85]	0.0001	0.0002	equivalent
H7a	H7, 1,060	PRM, W4, ±3	64.2	63.6	55.9	−0.57 [−1.32, +0.19]	$\leq 10^{-4}$	≤0.0004	equivalent
H7b	H7, 1,060	8B ORM, $R > 0$				+1.23 [−0.14, +2.59]	0.041	0.041	supported
H7b	H7, 1,060	PRM, $R > 0$				+3.54 [+2.45, +4.72]	$\leq 10^{-4}$	≤0.0003	supported

4.1. Four-bit quantization preserves Best-of-32 accuracy

The registered contrast c1 (8B ORM, GPTQ-W4, original 1,200-problem manifest, BF16 accuracy 77.5%) is $\Delta_{32} = +0.33$ pp with a 90% interval of $[-0.58, +1.25]$, inside the registered ±3 pp margin ($p_{\mathrm{TOST}} \leq 10^{-4}$, which passes any Holm threshold of the family): **equivalence is supported**. On

this population a 3 pp loss would have meant 13% more errors. On the full 2,000 problems the same contrast is +0.15 pp [−0.65, +0.95] (95% interval; Table 3).

Table 3 gives the exploratory grid on the 2,000 problems. At 4 bits, GPTQ-W4 and NF4 change A_{32} by between −1.1 and +0.6 pp for the three ORMs and the PRM, while the selected candidate changes on 21–49% of problems; the 4-bit curves track BF16 at every budget $N \leq 32$ (Fig. 1). Three bits is where selection degrades with this recipe: GPTQ-W3 costs 1.3–1.8 pp for the 4B, 8B and PRM verifiers and 13.2 pp for the 1.7B ORM, whose selection falls towards the random-candidate rate. The quantization method matters little at 4 bits for the 4B and 8B ORMs (AWQ-W4: 0.00 and +0.55 pp; round-to-nearest RTN-W4: +0.25 and +0.90 pp; RTN-W8 on the 8B: +0.25 pp), but the 1.7B ORM is sensitive to it: RTN-W4 costs it 2.8 pp [−4.0, −1.7] against 1.1 pp for GPTQ-W4.

4.2. Under the 6 GB verifier budget, the tested 4B 4-bit verifier is best

Fig. 2 shows accuracy against measured verifier memory for the tested Skywork ORM deployments. At the registered 6 GB ceiling, the configuration selected on development data is the 4B ORM with packed GPTQ-W4 (4.9 GiB peak), and the best BF16 verifier that fits is the 1.7B ORM (4.0 GiB). (The plan states the ceiling as 6 GB and it was applied to the GiB measurements; the selected configuration, 4.9 GiB = 5.3 GB, fits either reading.) The registered contrast c2 is +2.25 pp [+1.05, +3.45], $p \leq 10^{-4}$, Holm $p \leq 0.0005$: **supported**. The 1.7B BF16 verifier is 0.3 pp below majority vote, which needs no verifier memory at all, whereas the 4B GPTQ-W4 verifier is 2.0 pp above it. The comparison covers the tested checkpoints, a fixed scoring

Table 3: Best-of-32 accuracy (%) and change versus BF16 (pp, 95% CI), main study (2,000 problems, Qwen2.5-1.5B-Instruct candidates). Winner change: share of problems whose selected candidate differs from BF16's. Verifier-free baselines: majority vote 72.1%, oracle pass@32 86.9%, random candidate 52.1%.

verifier	precision	A_{32}	Δ_{32} [95% CI]	winner change
Skywork-1.7B	BF16	71.8	–	–
	NF4	70.7	−1.10 [−2.00, −0.25]	34%
	GPTQ-W4	70.7	−1.10 [−2.25, +0.05]	49%
	GPTQ-W3	58.7	−13.15 [−15.05, −11.25]	88%
Skywork-4B	BF16	73.5	–	–
	NF4	72.7	−0.80 [−1.55, −0.10]	24%
	GPTQ-W4	74.1	+0.55 [−0.30, +1.40]	29%
	GPTQ-W3	71.8	−1.75 [−3.05, −0.45]	60%
Skywork-8B	BF16	72.9	–	–
	NF4	72.9	−0.05 [−0.75, +0.65]	21%
	GPTQ-W4	73.1	+0.15 [−0.65, +0.95]	25%
	GPTQ-W3	71.3	−1.60 [−2.75, −0.45]	50%
PRM-7B	BF16	76.6	–	–
	NF4	76.5	−0.05 [−0.65, +0.50]	25%
	GPTQ-W4	76.3	−0.30 [−1.05, +0.45]	33%
	GPTQ-W3	75.3	−1.30 [−2.30, −0.25]	54%

workload and verifier memory only; within this family, accuracy does not grow with size (the 4B BF16 model beats the 8B), so the result is about these deployments rather than a general rule that larger is better.

Part of the advantage is answer format rather than answer content. With

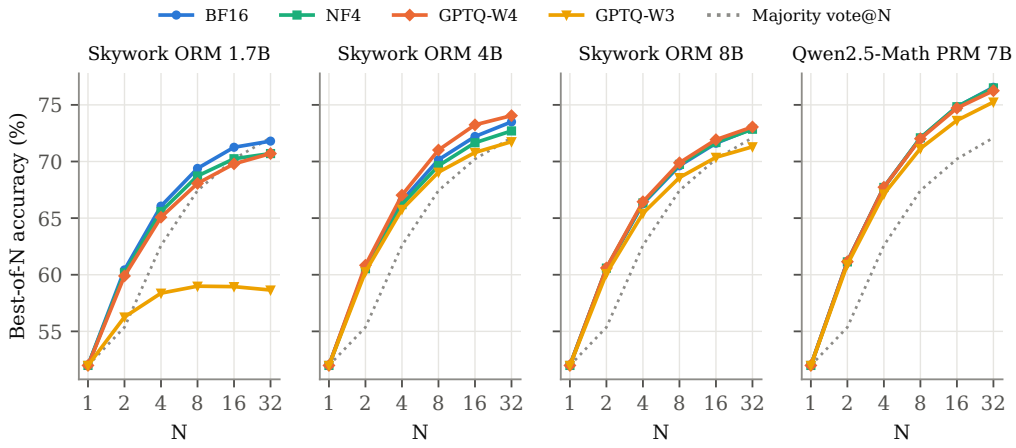


Figure 1: Best-of-N accuracy as a function of the number of candidates N, per verifier and precision (main study, 2,000 problems; 200 random subsets per problem for $N < 32$). The dotted line is the verifier-free majority vote over the same N candidates.

the lenient grader, c2 is $+1.55$ pp $[+0.45, +2.65]$. As a secondary analysis, one of the authors graded, blind to arm, verifier and grader verdict, every distinct Best-of-32 winner without an extractable final answer in either arm (94 responses) and 30 random winners with one (Appendix B); identical responses selected in both arms received a single label. Replacing the strict grader by these labels on all audited winners gives $+1.40$ pp $[+0.30, +2.50]$, nominal $p = 0.011$ (4B GPTQ-W4 75.5% versus 1.7B BF16 74.1%); this interval treats the human labels as error-free and the analysis is not part of the registered family. The registered strict-grader effect is therefore an upper estimate, and we regard $+1.4$ pp as the more realistic size. It does not depend on label noise or truncation: with GSM8K labels replaced by GSM8K-Platinum it is $+2.37$ pp, and with length-truncated candidates removed before selection $+2.20$ pp (strict grader).

A re-measurement on another RTX 4090 (exploratory) reproduced the registered peak memory of every Skywork deployment to within 0.01 GiB and added the PRM: 16.7 GiB in BF16 and 8.0 GiB with deployed NF4, whose scores are identical to the simulated NF4 condition. With the PRM included, the best tested deployment under an 8 GiB budget is the NF4 PRM (76.5%), while the 6 GB comparison of c2 is unchanged. Four-bit weights did not slow scoring with this workload: throughput was within 1–4% of BF16 for every model (e.g. 8B ORM: 9.8 sequences/s in BF16, 9.7 with packed GPTQ-W4 and with NF4).

4.3. Selected candidates change often; selected answers rarely do

Table 4 decomposes the 4-bit and 3-bit changes into answer transitions. With GPTQ-W4 or NF4 the selected candidate changes on 21–49% of main-study problems, but the selected canonical answer changes on only 4.5–12.8%: 73–81% of the candidate changes keep the same final answer. Harm (a correct BF16 selection becomes wrong) and benefit (the reverse) are each 0.9–4.1% of problems and largely offset each other. At 3 bits both the answer-change rate (12.6–37.8%) and harm grow, and harm exceeds benefit. The same pattern holds in H6 and H7. These measurements are consistent with answer redundancy: candidates that share the selected answer are interchangeable for accuracy, so most score perturbations move the selection within the same answer cluster.

4.4. No confirmatory evidence for the MATH-loss and budget-growth contrasts

Development data had suggested that quantization damage concentrates on MATH and grows with N. On the 1,200 evaluation MATH problems, c4

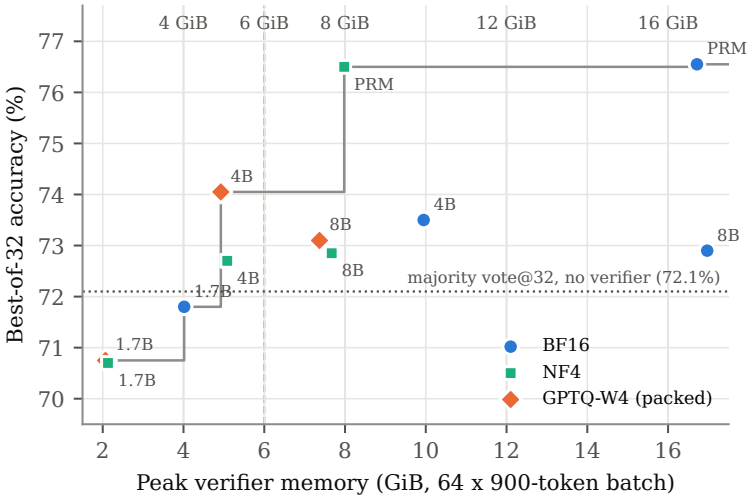


Figure 2: Accuracy against measured verifier memory for the tested Skywork ORM and PRM deployments (Best-of-32, main study, 2,000 problems; PRM memory from the exploratory re-measurement). Memory is the peak allocation while scoring a fixed workload (64 sequences of 900 tokens, RTX 4090, no generator); GPTQ-W4 uses packed 4-bit kernels and NF4 `bitsandbytes`. Vertical lines mark the registered ceilings, the dashed one the primary 6 GB ceiling of c2 (applied to GiB). The step line is the best accuracy among the tested deployments within each budget; the dotted line is majority vote, which uses no verifier.

(MATH Δ_{32}, 4B and 8B ORMs at GPTQ-W4, averaged within problem) is -0.63 pp $[-1.38, +0.08]$, Holm $p = 0.40$; c5 (the change of that loss from $N = 4$ to $N = 32$) is -0.51 pp $[-1.20, +0.16]$, Holm $p = 0.42$. Neither is supported. These are inconclusive results, not evidence of no effect; the intervals exclude MATH losses larger than about 1.4 pp. c4 is a within-MATH contrast, not a test of an interaction with difficulty; the development signal came from 200 problems and did not replicate.

Table 4: Answer transitions at $N = 32$ relative to BF16 (% of problems). Cand.: selected candidate changes; answer: selected canonical answer changes; same: share of candidate changes that keep the answer; harm: correct → wrong; benefit: wrong → correct; Δ_{32} = benefit − harm (pp).

study	verifier	condition	cand.	answer	same	harm	benefit	Δ_{32}
main	ORM 1.7B	GPTQ-W4	48.9	12.8	74	4.1	3.0	−1.10
		GPTQ-W3	87.6	37.8	57	17.7	4.5	−13.15
	ORM 4B	GPTQ-W4	29.2	7.8	73	1.7	2.3	+0.55
		GPTQ-W3	59.7	17.2	71	5.2	3.5	−1.75
	ORM 8B	BF16-null	6.9	0.8	88	0.3	0.2	−0.10
		NF4	21.2	4.5	79	1.3	1.2	−0.05
		GPTQ-W4	25.1	5.9	76	1.5	1.7	+0.15
		GPTQ-W3	50.5	13.5	73	4.3	2.7	−1.60
	PRM	NF4	25.2	4.8	81	0.9	0.9	−0.05
		GPTQ-W4	33.0	6.2	81	1.6	1.3	−0.30
		GPTQ-W3	54.0	12.6	77	3.4	2.1	−1.30
H6	ORM 8B	GPTQ-W4	31.4	6.6	79	1.3	1.3	0.00
	PRM	GPTQ-W4	34.4	5.4	84	0.8	1.3	+0.50
H7	ORM 8B	GPTQ-W4	26.9	8.4	69	2.1	1.8	−0.28
	PRM	GPTQ-W4	30.1	8.1	73	1.0	0.5	−0.57
	ORM 8B	GPTQ-W3	51.7	20.0	61	5.8	2.3	−3.49
	PRM	GPTQ-W3	55.9	18.2	67	3.8	1.2	−2.55

4.5. A stronger generator on hard problems (H6)

H6 asked whether the result survives a stronger generator on harder, previously unused problems: Qwen2.5-Math-7B-Instruct [48], 800 MATH Level-5 test problems, 32 candidates with up to 2,048 tokens, the 8B ORM

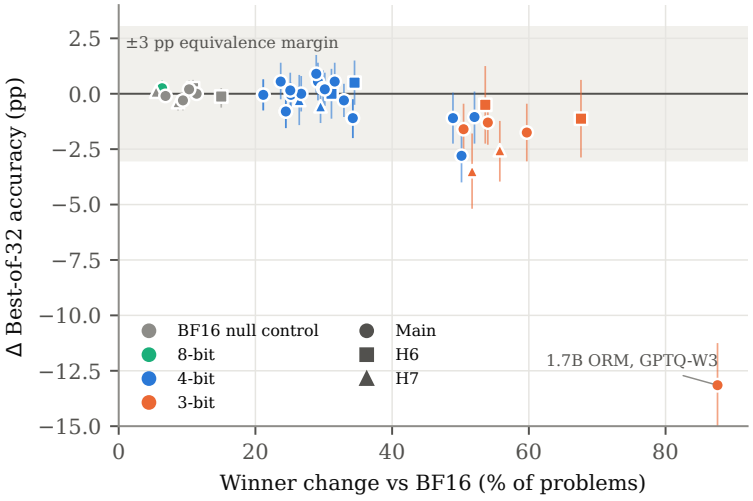


Figure 3: Winner change versus accuracy change at $N = 32$ for every quantized condition and control in the main study (circles) and the two follow-up studies (H6 squares, H7 triangles), with 95% bootstrap intervals. Four-bit conditions change the selected candidate on 21–52% of problems with point estimates inside the ±3 pp band; 3-bit conditions change about half or more of the winners and lose accuracy.

and the PRM in BF16 and GPTQ-W4. Its family of four tests asked for equivalence at ±3 pp (H6a) and for retention of at least half of the BF16 verifier's advantage over majority vote (H6b). With $G = A_{\text{BF16}} - A_{\text{MV}}$ the BF16 advantage and Δ the quantization effect, H6b tests

$$R \;=\; \tfrac{1}{2}G + \Delta \;=\; (A_{\text{Q}} - A_{\text{MV}}) - \tfrac{1}{2}G \;>\; 0,$$

i.e. the quantized verifier's advantage exceeds half of the BF16 advantage. R is computed per problem, $R_i = C_{\text{Q},i} - \frac{1}{2}(C_{\text{BF16},i} + C_{\text{MV},i})$, and bootstrapped as a mean. $R > 0$ implies retained value only when $G > 0$; the plans therefore interpret H6b/H7b only for verifiers whose BF16 advantage is positive, which we check descriptively with 97.5% intervals.

Equivalence is supported for both verifiers: $\Delta_{32} = 0.00$ pp $[-1.13, +1.13]$ for the ORM and $+0.50$ pp $[-0.50, +1.50]$ for the PRM, although the selected candidate changes on 31% and 34% of problems. With this generator, however, neither verifier showed a clear positive advantage over majority vote (76.5%): the PRM tied it ($G = +0.13$ pp $[-1.88, +2.13]$) and the ORM fell below it ($G = -3.75$ pp $[-6.25, -1.25]$). H6 therefore did not establish a positive BF16 advantage to retain, which limits the practical interpretation of its retention tests (ORM $R = -1.88$ pp, PRM $+0.56$ pp; neither supported). These conclusions hold under the lenient grader and without truncated candidates (19% of candidates reached the length limit).

4.6. Value retention where the verifier adds value (H7)

H7 is a prospective, targeted follow-up in a regime selected with earlier exploratory data: the main generator on hard problems, where the main-study PRM and 8B ORM had beaten majority vote. It uses all 1,060 MATH Level-4 and Level-5 test problems not used anywhere else in this work, the registered generation settings, and the same verifiers, conditions, tests and analysis code as H6. Because the regime was chosen with knowledge of earlier results, H7 estimates the effect of quantization in that regime, not the typical value of a verifier across deployments.

Both verifiers beat majority vote in BF16 on the new problems: the PRM by $+8.2$ pp (97.5% interval $[+5.9, +10.7]$) and the 8B ORM by $+3.0$ pp $[+0.4, +5.8]$. All four registered hypotheses are supported (Holm over four). Accuracy is equivalent at ± 3 pp: $\Delta_{32} = -0.28$ pp $[-1.42, +0.85]$ for the ORM and -0.57 pp $[-1.32, +0.19]$ for the PRM. The quantized verifiers keep more than half of their BF16 advantage: decisively for the PRM ($R = +3.54$ pp

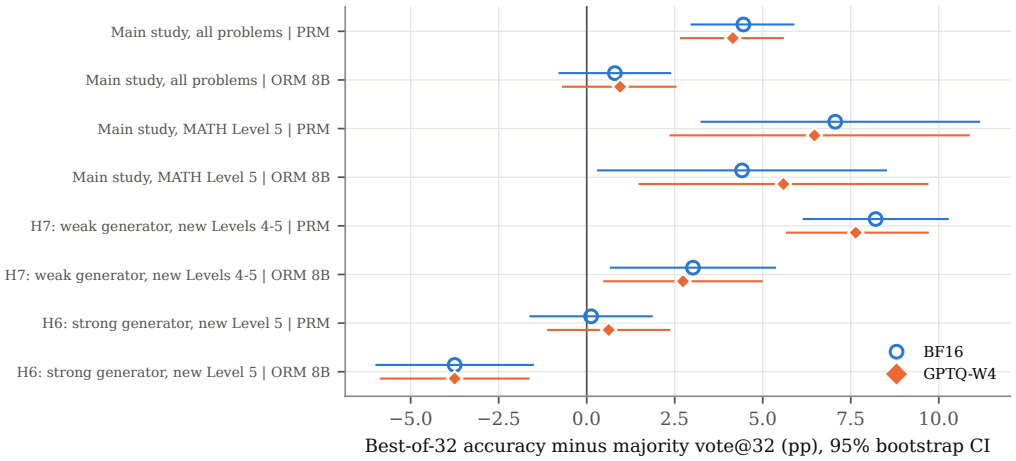


Figure 4: Verifier advantage over majority vote (Best-of-32 accuracy minus majority vote@32, pp, with 95% bootstrap intervals) in BF16 (open circles) and with GPTQ-W4 (diamonds), for the 8B ORM and the PRM. Main-study rows are exploratory subsets; H6 and H7 are separately planned studies on new problems.

$[+2.45, +4.72]$, Holm $p \leq 0.0003$), narrowly for the ORM ($R = +1.23$ pp $[-0.14, +2.59]$, one-sided Holm $p = 0.041$), in line with its lower planned power. As descriptive quantities, the PRM retains 93% of its advantage (95% interval 84–102%) and the ORM 91% (44–149%; the ratio is unstable because the ORM's advantage is small). The retention conclusions keep their sign under the lenient grader and without truncated candidates, the registered condition for describing value as preserved. Three bits is not safe in this regime (exploratory): GPTQ-W3 costs 3.5 pp (ORM) and 2.6 pp (PRM) and removes the ORM's advantage. Level-wise estimates and all sensitivity analyses are in the repository. Fig. 4 summarises the advantage over majority vote before and after quantization across regimes.

4.7. Robustness and controls

Other reward-model families. Two further ORMs on a Llama 3.1 backbone were evaluated on the main and H7 pools (exploratory). The 4-bit result holds within the ±3 pp margin, but with somewhat larger losses than for the Skywork-Qwen models. For Llama-3.1-Tulu-3-8B-RM, GPTQ-W4 changes A_{32} by -0.60 pp $[-1.65, +0.45]$ on the main problems and -0.75 pp $[-2.45, +0.85]$ on H7; for Skywork-Reward-V2-Llama-3.1-8B by -1.45 pp $[-2.75, -0.15]$ and -1.23 pp $[-3.11, +0.66]$. All four 90% intervals lie inside ±3 pp, but the Skywork-Llama main-study loss is distinguishable from zero. Both models changed the selected candidate (44–53%) and the selected answer (11–24%) more often than the Skywork-Qwen ORMs. Neither beat majority vote in BF16 (Tulu: $+0.35$ pp on the main problems, -0.94 pp on H7; Skywork-Llama: -3.35 and -2.17 pp), so their value retention cannot be assessed. At 3 bits with generic calibration both Llama-based models break down: GPTQ-W3 costs Tulu-3 12.6 pp $[-14.4, -10.8]$ and Skywork-Llama 26.4 pp $[-28.7, -24.1]$ (below the random-candidate rate), against 1.6 pp for the Qwen-based 8B ORM, with the selected answer changing on 34% and 57% of problems. The 3-bit behaviour is therefore strongly model-dependent.

Task-matched calibration at 3 bits. Much of the 3-bit loss comes from the generic calibration data (exploratory). Calibrating GPTQ-W3 on 128 candidates sampled from the generator's development pools instead of WikiText changes Δ_{32} from -13.15 to -3.75 pp $[-5.25, -2.30]$ for the 1.7B ORM (a recovery of $+9.4$ pp $[+7.6, +11.3]$) and from -1.60 to -0.05 pp $[-1.00, +0.90]$ for the 8B ORM ($+1.55$ pp $[+0.40, +2.75]$). The calibration problems are development problems, disjoint from the evaluation set. The 3-bit boundary

we observe is therefore a property of the recipe, not only of the bit width.

Selection rules. The 4-bit result does not depend on the selection rule (exploratory, main study, $N = 32$): with a reward-weighted answer vote (temperature fitted on development data), GPTQ-W4 changes accuracy by -1.00, -0.60, $+0.30$ and $+0.25$ pp for the 1.7B, 4B and 8B ORMs and the PRM; restricting Best-of-32 to candidates with an extractable answer gives -1.10, -0.55, 0.00 and -0.45 pp; and the PRM's last-step aggregation gives -0.25 pp. GPTQ-W3 again loses 0.7–11 pp under every rule. Restricting selection to extractable answers raises the BF16 ORMs' accuracy by 2.2–3.2 pp (e.g. 8B: 72.9% to 76.1%), a measure of how much the strict grader penalises their preference for prose answers.

Answer format. Several quantized conditions appeared to *improve* GSM8K accuracy under the strict grader (e.g. 4B GPTQ-W4 +3.0 pp; +8.0 pp on the Llama pools). A decomposition into format and content shows these are format effects: quantization weakens the Skywork ORMs' preference for prose answers without a boxed final answer (4B GPTQ-W4: format term +3.0, content term 0.0; lenient +0.4 pp). MATH effects are content effects under both graders. We report no quantization gain.

Packed versus simulated quantization. Packed and simulated GPTQ-W4 select the same final answer on 98.8–99.2% of problems; correctness flips are rare and symmetric (0.35–0.55% of problems), and packed GPTQ-W4 changes accuracy versus BF16 by -1.05, $+0.55$ and $+0.20$ pp for the 1.7B, 4B and 8B ORMs (simulated: -1.10, $+0.55$, $+0.15$). c2 uses the packed scores.

Second generator. On 300 problems with Llama-3.2-3B-Instruct candidates (exploratory), GPTQ-W4 changes the PRM's Best-of-32 accuracy by +0.33 pp $[-3.00, +4.00]$ and NF4 by −0.67 pp; for the 4B ORM the strict grader shows +8.0 pp, which shrinks to +2.0 pp $[0.0, +4.3]$ under the lenient grader, again a format effect. With this generator, majority vote (76.3%) beats both verifiers in BF16 (ORM 63.7%, PRM 70.7%).

Labels and truncation. With GSM8K-Platinum labels c1 is +0.44 pp (c4 and c5 involve MATH only and are unchanged); with truncated candidates removed before selection, c1 is +0.33, c4 −0.67 and c5 −0.52 pp. No conclusion changes. Both GSM8K problems whose labels the human audit had flagged as questionable are among those GSM8K-Platinum removes.

5. Discussion

Deployment implications. For the tested reward-model verifiers of 4B parameters and above, 4-bit weight-only GPTQ preserved Best-of-N accuracy in three prospectively planned settings (two generators, three problem sets, an ORM family and a PRM), and where the verifier beat majority vote it kept most of that advantage. NF4 and AWQ behaved similarly in the exploratory grid but were not part of the follow-up studies. Two reward models on a Llama backbone also stayed within the margin, but lost 0.6–1.5 pp and changed their selections more often, so the size of the 4-bit effect varies between models even when it is small. Under the tested 6 GB verifier budget and workload, a 4B ORM with packed 4-bit weights was the best tested deployment, ahead of the best full-precision verifier that fit. Three bits was not robust with generic calibration: it cost 1.3–1.8 pp for the larger verifiers in the main study,

2.6–3.5 pp in H7, and 13.2 pp for the 1.7B ORM, which was also sensitive to the quantization method at 4 bits (RTN versus GPTQ), and 12.6–26.4 pp for the two Llama-based reward models. Calibrating on in-domain candidates instead of generic text removed most of this 3-bit loss in an exploratory test (8B ORM: -0.05 pp; 1.7B: -3.75 instead of -13.15 pp), so the calibration data matter as much as the bit width at 3 bits. These conclusions are specific to the tested checkpoints, quantization recipes, scoring rule and mathematical tasks.

Why accuracy is stable when selections change. Quantization perturbs verifier scores enough to change the selected candidate on a quarter to a half of problems at 4 bits, yet the selected canonical answer changes far less often (Table 4): most changes stay within the same answer cluster, and the remaining harms and benefits largely offset each other. This is consistent with answer redundancy in the pools: Best-of-N needs the top of the ranking to keep a correct answer, not a particular candidate. We have not isolated this mechanism causally (for example by manipulating answer multiplicities), and the remaining answer-level changes are concentrated where competing answers have similar support. It suggests that quantization sensitivity should depend on how often answers compete closely rather than on nominal difficulty or on N, which agrees with the unsupported difficulty and budget contrasts (c4, c5).

How much a verifier is worth. Asking whether quantization destroys the value a verifier adds presupposes that the verifier adds value. With the main generator the Skywork ORMs were within about one point of majority vote overall; with a strong math generator on hard problems majority vote matched

the PRM and beat the ORM; only in the regime selected for verifier advantage (H7) did both verifiers clearly beat majority vote. Aggregation rules that combine answer frequency and reward scores can change this comparison [19]; our weighted-vote control shows the same robustness to quantization under such a rule. Practitioners should compare a verifier against self-consistency on their own generator before paying its memory and latency cost.

Evaluating selection. Three practices changed our conclusions or their size. (i) A strict answer grader turned a reward-model preference for prose answers into apparent quantization "gains" of up to 8 pp; a format/content decomposition and a lenient grader removed them, and a blinded human audit of the unparseable winners reduced the registered memory-budget effect from 2.25 to 1.4 pp. (ii) A fixed-precision null control (same weights, different batch layout) changes 5–15% of selected candidates across our studies; winner-change rates without such a reference overstate instability. (iii) Development-set signals that motivated two registered hypotheses did not replicate on 1,200 evaluation problems; registering them made the non-replication visible.

Limitations. The main-study ORMs come from a single family (Skywork-Reward-V2, built on Qwen3), the PRM is a Qwen model and the generators are Qwen and Llama models; the two additional Llama-based reward models address the backbone and training-family question only in an exploratory way, and neither added value over majority vote, so value retention outside the Qwen-based verifiers is untested. Accuracy experiments use simulated quantization; packed 4-bit kernels agreed with it on 98.8–99.2% of selected answers, and memory was measured for one scoring workload without the generator. The confirmatory GPTQ results use generic calibration with a

single calibration draw; in-domain calibration was tested only at 3 bits and for two verifiers. The tasks are GSM8K and MATH, whose test problems may have been seen during pre-training of the generators and verifiers; because BF16 and quantized verifiers score the same frozen pools, exposure is shared by design, but it may alter pool redundancy and verifier advantage and could interact with quantization. Candidates were capped at 1,024 or 2,048 tokens and 6.5–19% reached the cap, although excluding them changes no conclusion. The human audit was performed by a single rater on the unparseable winners only, and its corrected estimate treats the labels as error-free. The value-retention result is confirmatory for the PRM and only narrowly so for the ORM, and it holds in a regime chosen with prior knowledge; we did not find a strong-generator regime in which the verifiers clearly beat majority vote. Finally, the analysis plans were fixed in version control without an external registry, and the main-study plan was amended after candidate generation but before any verifier scoring.

6. Conclusion

We asked whether the reward-model verifier in Best-of-N mathematical answer selection can be quantized without losing what it is for. On frozen candidate pools, with prospectively specified analysis plans, the tested 4-bit GPTQ deployments of verifiers with 4B parameters and above kept Best-of-32 accuracy within a ± 3 pp equivalence margin in the main study and in two follow-up studies, and where the verifier beat majority vote they kept more than half of that advantage (an estimated 93% for the PRM). Under the tested 6 GB verifier budget, the 4B ORM with packed 4-bit weights was the

best tested deployment. Quantization changes the selected candidate often, but the selected answer much less often, consistent with answer redundancy in the pools. Three-bit GPTQ with generic calibration degrades selection, most for the smallest verifier, but in-domain calibration recovers most of the loss. Two practical caveats travel with these results. Compare a verifier against majority vote on one's own generator, because with a strong generator it may add little; and evaluate selection with a grader that is robust to answer format, because a strict grader can turn format preferences into spurious effects of quantization.

CRediT authorship contribution statement

Hector Borobia: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Elies Seguí-Mas:** Writing – review & editing, Supervision. **Guillermina Tormo-Carbó:** Writing – review & editing, Supervision.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used Claude Code (Anthropic) to support code development, language editing, and consistency checking. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Code, configuration, frozen problem manifests, analysis plans with their amendments, analysis scripts and figure-generation scripts are available at the project repository: [TODO: https://github.com/hecboar/quantized-verifiers-bon (tag v1.0-submission)]. The candidate pools and all verifier scores are available at [TODO: Hugging Face dataset URL].

References

[1] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, J. Schulman, Training verifiers to solve math word problems, arXiv preprint arXiv:2110.14168 (2021).

[2] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, K. Cobbe, Let's verify step by step,

in: The Twelfth International Conference on Learning Representations (ICLR), 2024, arXiv:2305.20050.

[3] C. Snell, J. Lee, K. Xu, A. Kumar, Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning, in: The Thirteenth International Conference on Learning Representations (ICLR), 2025, preprint title: "Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters", arXiv:2408.03314.

[4] E. Frantar, S. Ashkboos, T. Hoefler, D. Alistarh, GPTQ: Accurate post-training quantization for generative pre-trained transformers, in: The Eleventh International Conference on Learning Representations (ICLR), 2023, published at ICLR 2023 as "OPTQ: Accurate Quantization for Generative Pre-trained Transformers"; arXiv:2210.17323.

[5] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, S. Han, AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration, in: Proceedings of Machine Learning and Systems, Vol. 6, 2024, arXiv:2306.00978.

[6] T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer, QLoRA: Efficient finetuning of quantized LLMs, in: Advances in Neural Information Processing Systems, Vol. 36, 2023, pp. 10088–10115.

[7] L. Gao, J. Schulman, J. Hilton, Scaling laws for reward model overoptimization, in: Proceedings of the 40th International Conference on Machine Learning (ICML), Vol. 202 of Proceedings of Machine Learning Research, PMLR, 2023, pp. 10835–10866.

[8] P. Wang, L. Li, Z. Shao, R. X. Xu, D. Dai, Y. Li, D. Chen, Y. Wu, Z. Sui, Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations, arXiv preprint arXiv:2312.08935 (2023). arXiv:2312.08935.

[9] A. Setlur, C. Nagpal, A. Fisch, X. Geng, J. Eisenstein, R. Agarwal, A. Agarwal, J. Berant, A. Kumar, Rewarding Progress: Scaling Automated Process Verifiers for LLM Reasoning, arXiv preprint arXiv:2410.08146 (2024). arXiv:2410.08146.

[10] Z. Zhang, C. Zheng, Y. Wu, B. Zhang, R. Lin, B. Yu, D. Liu, J. Zhou, J. Lin, The lessons of developing process reward models in mathematical reasoning, in: Findings of the Association for Computational Linguistics: ACL 2025, Association for Computational Linguistics, Vienna, Austria, 2025, pp. 10495–10516, arXiv:2501.07301. doi:10.18653/v1/2025.findings-acl.547.

[11] L. Zhang, A. Hosseini, H. Bansal, M. Kazemi, A. Kumar, R. Agarwal, Generative Verifiers: Reward Modeling as Next-Token Prediction, arXiv preprint arXiv:2408.15240 (2024). arXiv:2408.15240.

[12] R. Liu, J. Gao, J. Zhao, K. Zhang, X. Li, B. Qi, W. Ouyang, B. Zhou, Can 1B LLM Surpass 405B LLM? Rethinking Compute-Optimal Test-Time Scaling, arXiv preprint arXiv:2502.06703 (2025). arXiv:2502.06703.

[13] N. Singhi, H. Bansal, A. Hosseini, A. Grover, K.-W. Chang, M. Rohrbach, A. Rohrbach, When To Solve, When To Verify: Compute-Optimal

Problem Solving and Generative Verification for LLM Reasoning, arXiv preprint arXiv:2504.01005 (2025). arXiv:2504.01005.

[14] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, D. Zhou, Self-consistency improves chain of thought reasoning in language models, in: The Eleventh International Conference on Learning Representations (ICLR), 2023, arXiv:2203.11171.

[15] B. Brown, J. Juravsky, R. Ehrlich, R. Clark, Q. V. Le, C. Ré, A. Mirhoseini, Large Language Monkeys: Scaling Inference Compute with Repeated Sampling, arXiv preprint arXiv:2407.21787 (2024). arXiv:2407.21787.

[16] Y. Wu, Z. Sun, S. Li, S. Welleck, Y. Yang, Inference Scaling Laws: An Empirical Analysis of Compute-Optimal Inference for Problem-Solving with Language Models, arXiv preprint arXiv:2408.00724 (2024). arXiv:2408.00724.

[17] L. Yuan, W. Li, H. Chen, G. Cui, N. Ding, K. Zhang, B. Zhou, Z. Liu, H. Peng, Free Process Rewards without Process Labels, arXiv preprint arXiv:2412.01981 (2024). arXiv:2412.01981.

[18] Q. Di, K. Ji, X. Li, H. Zhao, Q. Gu, Best-of-Majority: Minimax-Optimal Strategy for Pass@k Inference Scaling, arXiv preprint arXiv:2510.03199 (2025). arXiv:2510.03199.

[19] P. Kuang, Y. Wang, X. Han, Y. Liu, K. Xu, H. Wang, Optimal aggregation of LLM and PRM signals for efficient test-time scaling, arXiv preprint arXiv:2510.13918 (2025).

[20] J. Kim, E. Ewer, T. Moon, J. Park, D. Papailiopoulos, Not All Bits Are Equal: Scale-Dependent Memory Optimization Strategies for Reasoning Models, arXiv preprint arXiv:2510.10964 (2025). arXiv:2510.10964.

[21] H. Sun, M. Haider, R. Zhang, H. Yang, J. Qiu, M. Yin, M. Wang, P. Bartlett, A. Zanette, Fast best-of-N decoding via speculative rejection, in: Advances in Neural Information Processing Systems, 2024, arXiv:2410.20290.

[22] J. Guo, Z. Wu, H. Yang, P. S. Yu, Mining intrinsic rewards from LLM hidden states for efficient best-of-N sampling, arXiv preprint arXiv:2505.12225 (2025).

[23] P. Kuang, H. Jin, X. Han, Y. Wang, X. Yuan, Y. Yu, K. Xu, H. Wang, KV-PRM: Efficient process reward modeling via KV-cache transfer for multi-agent test-time scaling, arXiv preprint arXiv:2607.09153 (2026).

[24] N. Lambert, V. Pyatkin, J. Morrison, L. Miranda, B. Y. Lin, K. Chandu, N. Dziri, S. Kumar, T. Zick, Y. Choi, N. A. Smith, H. Hajishirzi, RewardBench: Evaluating Reward Models for Language Modeling, arXiv preprint arXiv:2403.13787 (2024). arXiv:2403.13787.

[25] C. Zheng, Z. Zhang, B. Zhang, R. Lin, K. Lu, B. Yu, D. Liu, J. Zhou, J. Lin, ProcessBench: Identifying Process Errors in Mathematical Reasoning, arXiv preprint arXiv:2412.06559 (2024). arXiv:2412.06559.

[26] M. Song, Z. Su, X. Qu, J. Zhou, Y. Cheng, PRMBench: A Fine-grained and Challenging Benchmark for Process-Level Reward Models, arXiv preprint arXiv:2501.03124 (2025). arXiv:2501.03124.

[27] T. Dettmers, M. Lewis, Y. Belkada, L. Zettlemoyer, LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale, arXiv preprint arXiv:2208.07339 (2022). arXiv:2208.07339.

[28] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, S. Han, SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models, arXiv preprint arXiv:2211.10438 (2022). arXiv:2211.10438.

[29] T. Dettmers, L. Zettlemoyer, The case for 4-bit precision: k-bit Inference Scaling Laws, arXiv preprint arXiv:2212.09720 (2022). arXiv:2212.09720.

[30] E. Kurtic, A. Marques, S. Pandit, M. Kurtz, D. Alistarh, "Give Me BF16 or Give Me Death"? Accuracy-Performance Trade-Offs in LLM Quantization, arXiv preprint arXiv:2411.02355 (2024). arXiv:2411.02355.

[31] C. Zhu, H. Jiang, J. Liu, Y. Han, K. Zhao, R. Zhang, RotSQ: Joint quantization and sparsification of large language models in rotated space, Neurocomputing 707 (2027) 135039. doi:10.1016/j.neucom.2026.135039.

[32] W. Bang, J. Park, H. Yu, S. Han, K. Bin, K. Lee, BVSR: Bitvector-sum representation for distribution-aware LLM weight quantization, Neurocomputing 709 (2027) 135315. doi:10.1016/j.neucom.2026.135315.

[33] R. Liu, Y. Sun, M. Zhang, H. Bai, X. Yu, T. Yu, C. Yuan, L. Hou, Quantization Hurts Reasoning? An Empirical Study on Quantized Reasoning Models, arXiv preprint arXiv:2504.04823 (2025). arXiv:2504.04823.

[34] Z. Li, Y. Su, R. Yang, C. Xie, Z. Wang, Z. Xie, N. Wong, H. Yang, Quantization Meets Reasoning: Exploring LLM Low-Bit Quantization Degradation for Mathematical Reasoning, arXiv preprint arXiv:2501.03035 (2025). arXiv:2501.03035.

[35] A. Dutta, S. Krishnan, N. Kwatra, R. Ramjee, Accuracy is Not All You Need, arXiv preprint arXiv:2407.09141 (2024). arXiv:2407.09141.

[36] Anonymous, When BitsAndBytes changes the evaluator: Reliability of 8/4-bit LLM judges, NeurIPS 2026 workshop submission, OpenReview https://openreview.net/forum?id=3ChXlWOIrY (2026).

[37] H. Zhang, J. Da, D. Lee, V. Robinson, C. Wu, W. Song, T. Zhao, P. Raja, C. Zhuang, D. Slack, Q. Lyu, S. Hendryx, R. Kaplan, M. Lunati, S. Yue, A Careful Examination of Large Language Model Performance on Grade School Arithmetic, arXiv preprint arXiv:2405.00332 (2024). arXiv:2405.00332.

[38] J. Vendrow, E. Vendrow, S. Beery, A. Madry, Do large language model benchmarks test reliability?, arXiv preprint arXiv:2502.03461 (2025).

[39] A. Hochlehnert, H. Bhatnagar, V. Udandarao, S. Albanie, A. Prabhu, M. Bethge, A Sober Look at Progress in Language Model Reasoning: Pitfalls and Paths to Reproducibility, arXiv preprint arXiv:2504.07086 (2025). arXiv:2504.07086.

[40] S. Biderman, H. Schoelkopf, L. Sutawika, L. Gao, J. Tow, B. Abbasi, A. F. Aji, P. S. Ammanamanchi, S. Black, J. Clive, A. DiPofi, J. Etxaniz, B. Fattori, J. Z. Forde, C. Foster, J. Hsu, M. Jaiswal, W. Y. Lee, H. Li,

C. Lovering, N. Muennighoff, E. Pavlick, J. Phang, A. Skowron, S. Tan, X. Tang, K. A. Wang, G. I. Winata, F. Yvon, A. Zou, Lessons from the Trenches on Reproducible Evaluation of Language Models, arXiv preprint arXiv:2405.14782 (2024). arXiv:2405.14782.

[41] E. Miller, Adding Error Bars to Evals: A Statistical Approach to Language Model Evaluations, arXiv preprint arXiv:2411.00640 (2024). arXiv:2411.00640.

[42] J. M. Hofman, A. Chatzimparmpas, A. Sharma, D. J. Watts, J. Hullman, Pre-registration for Predictive Modeling, arXiv preprint arXiv:2311.18807 (2023). arXiv:2311.18807.

[43] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, J. Steinhardt, Measuring mathematical problem solving with the MATH dataset, in: Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks, Vol. 1, 2021, arXiv:2103.03874.

[44] Qwen Team, A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, et al., Qwen2.5 technical report, arXiv preprint arXiv:2412.15115 (2024).

[45] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, I. Stoica, Efficient memory management for large language model serving with PagedAttention, in: Proceedings of the 29th Symposium on Operating Systems Principles (SOSP), ACM, 2023, pp. 611–626. doi:10.1145/3600006.3613165.

[46] C. Y. Liu, L. Zeng, Y. Xiao, J. He, J. Liu, C. Wang, R. Yan, W. Shen, F. Zhang, J. Xu, Y. Liu, Y. Zhou, Skywork-Reward-V2: Scaling preference data curation via human-AI synergy, in: The Fourteenth International Conference on Learning Representations (ICLR), 2026, arXiv:2507.01352.

[47] N. Lambert, J. Morrison, V. Pyatkin, S. Huang, H. Ivison, F. Brahman, L. J. V. Miranda, A. Liu, N. Dziri, S. Lyu, et al., Tülu 3: Pushing frontiers in open language model post-training, arXiv preprint arXiv:2411.15124 (2024).

[48] A. Yang, B. Zhang, B. Hui, B. Gao, B. Yu, C. Li, D. Liu, J. Tu, J. Zhou, J. Lin, et al., Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement, arXiv preprint arXiv:2409.12122 (2024).

Appendix A. Implementation details

Prompt. Every generator receives the system prompt "Please reason step by step, and put your final answer within `\boxed{}`." and the problem as the user turn, through its chat template. For Llama-3.2-3B the instruction is repeated at the end of the user turn (registered deviation; without it 34% of its candidates had no extractable answer).

Verifier inputs. The ORMs (Skywork-Reward-V2-Qwen3 1.7B/4B/8B, Skywork-Reward-V2-Llama-3.1-8B, Llama-3.1-Tulu-3-8B-RM) receive the (problem, response) pair as a user/assistant conversation formatted by their chat template, without system prompt, with a single beginning-of-sequence

token, and output a scalar from their reward head; the padding token never occurs inside a sequence (checked). Tulu-3-8B-RM does not define a padding id in its configuration, so the tokenizer's padding token is set. The PRM receives its documented system prompt, the problem, and the response split into steps at blank lines, separated by its step token; the score is the minimum step reward and the last-step reward is stored for sensitivity analysis.

Quantization. All linear layers inside the transformer blocks are quantized (weight-only); embeddings, norms and the reward head stay in BF16, and activations stay in BF16. GPTQ uses group size 128, symmetric quantization and 1% diagonal damping, calibrated on 128 WikiText-2 sequences of 512 tokens drawn with a fixed seed. AWQ uses the same calibration and group size; NF4 uses `bitsandbytes` without double quantization; RTN uses group size 128 at 4 bits and per-channel scales at 8 bits. Every quantized model is audited by fingerprinting all parameters before and after quantization: every target layer must change (e.g. 252 of 252 for the 8B ORM, 196 of 196 for the PRM) and no other parameter may change.

PRM rotary buffers. With `transformers` 5.x, Qwen2.5-Math-PRM-7B's remote modelling code is instantiated on the meta device and its non-persistent rotary-embedding buffers come back uninitialised after loading. Scores computed this way were below the random-candidate rate. We recompute the buffers exactly as the model's own initialisation does and check them at every load; all PRM results in this paper use repaired buffers, and the affected scores were discarded (registered deviation).

Software and compute. Generation used vLLM 0.30.0; scoring used PyTorch 2.13.0 (CUDA 13.0) and `transformers` 5.17 (main study) or 5.19 (follow-up studies); packed kernels used `gptqmodel`. Model identifiers and all settings are recorded in the repository and in the metadata of every score file. All experiments ran on single cloud GPUs (mostly RTX 4090; some follow-up runs on L40S and other GPUs with at least 24 GB, with the BF16 and quantized runs of a verifier always on the same GPU), for roughly 90 GPU-hours in total including setup.

Appendix B. Blinded audit of the outcome measure

All 94 distinct Best-of-32 winners without an extractable final answer in either arm of the registered memory-budget contrast (c2), plus 30 random winners with one, were shown to a human rater (one of the authors) with the problem, the reference answer and the full response, rendered with mathematical typesetting, and nothing else. The endpoint was final-answer correctness; responses that reached the length limit were marked as such and counted as incorrect by a rule fixed beforehand, because they give no final answer. The rater judged 83 correct, 29 incorrect and 12 unsure. A blinded second pass (same view, key unopened) re-examined every unsure and incorrect verdict, resolved all 12 unsure verdicts as incorrect (9 with a wrong answer or reasoning, 3 cut by the length limit before giving an answer) and confirmed every incorrect one; it also flagged two GSM8K problems with questionable labels, both of which GSM8K-Platinum removes. With the first verdicts and unsure items dropped, the corrected c2 is $+1.41$ pp $[+0.35, +2.46]$ instead of $+1.40$. On the 30 controls the strict grader and the human agree

on 29. On the 94 winners without an extractable answer the strict grader scores all as wrong, the human judges 60 correct, and the lenient grader agrees with the human on 85. The audit covers only unparseable winners and was done by a single rater; residual grading error among parseable winners is not measured beyond the 30 controls.

Appendix C. Analysis plans and deviations

Table C.5 lists the chronology recorded in the repository. The deviations, all documented with dates, were: the Llama prompt repetition; the PRM rotary-buffer repair and rescoring; the frozen calibration hyper-parameter of the unrun c3 method (the registered decision rule permitted running c3, but at the registered 4 bits the 8B ORM showed no MATH damage to repair, so it was not run and stays in the family with $p = 1$); exploratory additions after the main results (bit-width dose-response, the 1.7B packed model in the frontier); in H6, the loss and regeneration of the first candidate pool (never inspected) and scoring split across four GPUs; in H7, GPUs other than the RTX 4090 for lack of availability and lenient-grader verdicts computed with time limits equivalent to those on Linux; corrections to the follow-up analysis script made before H7 data existed, which reproduce all H6 results exactly; and the exploratory revision experiments (additional reward models, task-matched 3-bit calibration, memory and throughput re-measurement), fixed before their launch.

Table C.5: Chronology of plans and data (commit timestamps, 2026).

date	event	data existing at that point
27 Sep	plan with c1–c3; evaluation and development manifests frozen	none
27–28 Sep	go/no-go gate and early signal on development problems	development scores only
28 Sep 11:20	amendment 1: c4, c5 and the 800-problem MATH extension	development scores only
28 Sep 11:43–14:00	main-study candidate pools generated	candidates (generator statistics inspected)
28 Sep 14:17	amendment 2: populations, test rules, c3 decision rule, pool hashes	candidates, no verifier score
28 Sep 14:17 onwards	main-study scoring and analysis	–
12 Oct 19:00	H6 plan, manifests and analysis code frozen	none for H6
13 Oct 02:10	H6 pool hashes recorded; scoring follows	H6 candidates
13 Oct 13:17	H7 plan frozen (same analysis code)	none for H7
13 Oct 14:27	H7 pool hashes recorded; scoring follows	H7 candidates
14 Oct 00:14	revision experiments fixed (exploratory)	no score of the new runs